

Operating System Project Final Report

Team 3

1. Introduction

This project simulates the core functions of an Operating System: **Scheduling** (deciding which program runs) and **Memory Management** (managing RAM). The goal is to model how a real OS handles multiple processes running at the same time with limited resources.

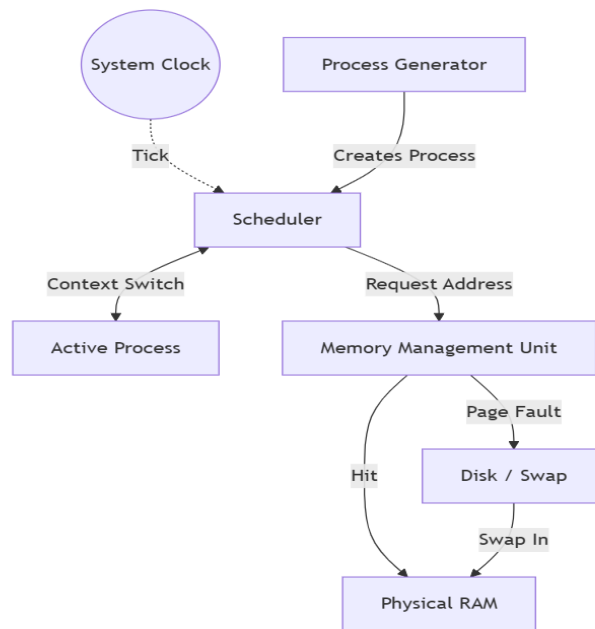
2. System Specifications

We simulate a small, constrained computer system to demonstrate OS concepts clearly.

- **CPU:** Single Core.
- **RAM:** 512 Bytes (Very small!).
- **Address Space:** 10-bit addressing (0 to 1023).
- **Page Size:** 16 Bytes.
- **Total Frames:** 32 physical frames available in RAM.
- **Disk Speed:** Loading data from disk takes **10 cycles** (slow) compared to instant RAM access.

3. System Architecture (Block Diagram)

The system is built from communicating modules that act like hardware and kernel components.



Component Roles

1. **Process Generator:** Reads the input file (processes.txt) and creates processes at the correct arrival time.
2. **Scheduler:** The "Brain" of the OS. It picks the next process to run using algorithms like Round Robin.
3. **MMU (Memory Management Unit):** Handles memory. It translates "Virtual Addresses" (used by the program) into "Physical Addresses" (in RAM). If data is missing, it fetches it from the disk.
4. **Clock:** Keeps global time for the simulation.

4. Phase 1: CPU Scheduling

The Scheduler ensures fairness and efficiency. We implemented three algorithms:

- **Round Robin (RR):** Each process gets a small slice of time (Quantum). If it doesn't finish, it goes to the back of the line. Used in Phase 2 for integration.
- **Highest Priority First (HPF):** The most important task runs first.
- **Shortest Remaining Time Next (SRTN):** The quickest task runs first.

5. Phase 2: Memory Management

Since RAM is small (only 32 frames), we cannot keep everything in memory. We use **Demand Paging**.

How it Works:

1. **Startup:** When a process starts, we only load its **Page Table** and **Page 0** (first code page) into RAM.
2. **Page Faults:** If a process needs a page not in RAM, it stops (Blocks). The OS fetches the page from disk (taking 10 cycles).
3. **Swapping (Second Chance):** If RAM is full, we must kick someone out. We use the **Second Chance Algorithm**:
 - We look at pages in a circle.
 - If a page was used recently (Reference Bit = 1), we give it a "Second Chance" and skip it.
 - If it wasn't used (Reference Bit = 0), we swap it out.
4. **Dirty Bits:** If the victim page was modified (Dirty), we save it to disk first (costing an extra 10 cycles).

6. Process States

A process moves through these states during its life:

- **READY:** Waiting for the CPU.
- **RUNNING:** Executing code.
- **BLOCKED_PAGE_FAULT:**
 - *I/O Block:* Waiting for a page to load from disk (10 cycles).
- **BLOCKED_DEPENDENCY:**
 - *Dependency Block:* Waiting for another process to finish.
- **TERMINATED:** Finished. Memory is freed.

7. Conclusion

This project demonstrates how an OS handles limited resources. By using **Virtual Memory**, we can run large processes on small RAM. By using **Scheduling**, we keep the CPU busy while processes wait for disk I/O. The simulation proves that efficient algorithms are needed to prevent the system from becoming slow due to constant disk access.